

W17D4 - Buffer Overflow in C e Prevenzione

1. Introduzione

L'obiettivo dell'esercitazione è dimostrare una vulnerabilità di tipo **Buffer Overflow (BOF)** in un programma scritto in C, causata dalla mancanza di controllo sull'input utente. Si mostra come un input eccessivo possa causare un **segmentation fault**, e come modifiche al codice possano prevenire il problema.

2. Preparazione dell'ambiente

2.1 Navigazione nel file system

Ci spostiamo nella directory Desktop per creare e compilare il file sorgente, è una posizione comoda per gestire file temporanei e test

Comando: `cd ~/Desktop`

3. Creazione del file sorgente vulnerabile

3.1 Creazione del file con nano

Apriamo l'editor di testo nano per scrivere il codice sorgente. Si apre un'interfaccia testuale dove inserire il codice C vulnerabile.

Comando: `nano BOF.c`

3.2 Codice iniziale vulnerabile

Codice:

```
#include <stdio.h>
```

```
int main () {  
    char buffer[10];  
    printf("Si prega di inserire il nome utente:");  
    scanf("%s", buffer);  
    printf("Nome utente inserito: %s\n", buffer);  
}
```

```
    return 0;
}
```

Il programma inizia includendo la libreria standard `stdio.h`, necessaria per gestire input e output. All'interno della funzione `main`, viene dichiarato un array di caratteri chiamato `buffer`, con una dimensione fissa di 10 byte. Questo buffer è destinato a contenere il nome utente inserito dall'utente. Subito dopo, il programma stampa a schermo un messaggio che invita l'utente a inserire il proprio nome.

La funzione `scanf` legge l'input da tastiera e lo memorizza nel buffer. Tuttavia, il formato utilizzato (`%s`) non impone alcun limite alla lunghezza dell'input, il che significa che l'utente può inserire una stringa molto più lunga dei 10 caratteri previsti. Se ciò accade, i dati in eccesso vengono scritti oltre i confini del buffer, sovrascrivendo aree di memoria non destinate a quell'uso. Questo comportamento porta a un errore di segmentazione, noto come `segmentation fault`, che interrompe bruscamente l'esecuzione del programma.

Infine, il programma stampa il nome utente inserito e termina con `return 0`, indicando che l'esecuzione è completata. Tuttavia, se l'input supera la capacità del buffer, il programma non arriva nemmeno a questa fase, perché viene interrotto dal sistema operativo a causa della violazione di memoria.



```
kali@kali: ~/Desktop
File Azioni Modifica Visualizza Aiuto
GNU nano 8.4 BOF.c
#include <stdio.h>

int main () {
    char buffer[10];
    printf("Si prega di inserire il nome utente:");
    scanf("%s", buffer);
    printf("Nome utente inserito: %s\n", buffer);
    return 0;
}
```

4. Compilazione del programma

4.1 Compilazione con GCC

Una volta scritto il codice sorgente, è necessario compilarlo per trasformarlo in un programma eseguibile. Per farlo, si utilizza il comando seguente, che invoca il compilatore GCC specificando il file da compilare (`BOF.c`) e il nome dell'eseguibile da generare (`BOF`). L'opzione `-g` viene aggiunta per includere le informazioni di debug, utili nel caso si voglia

analizzare il comportamento del programma con strumenti come gdb. Dopo l'esecuzione del comando, se il codice non presenta errori di sintassi, viene creato il file BOF, pronto per essere eseguito e testato. Questo passaggio è fondamentale per verificare il funzionamento del programma e osservare direttamente gli effetti del buffer overflow.

Comando: gcc -g BOF.c -o BOF

5. Esecuzione e test del programma vulnerabile

5.1 Esecuzione con input normale

Comando: ./BOF

Dopo aver compilato correttamente il programma, si procede con la sua esecuzione utilizzando il comando. All'avvio, il programma mostra a schermo il messaggio che invita l'utente a inserire il proprio nome. In questo primo test, si fornisce un input breve e controllato che rientra perfettamente nei limiti del buffer definito nel codice. Il programma riceve l'input, lo memorizza correttamente e lo restituisce a video senza alcun errore. Questo comportamento conferma che, in condizioni normali e con input validi, il programma funziona come previsto.

5.2 Esecuzione con input eccessivo

Comando: ./BOF

Inseriamo un nome utente con più caratteri e non avremo più lo stesso risultato

```
kali@kali: ~/Desktop
File Azioni Modifica Visualizza Aiuto
(kali@kali)-[~]
$ cd Desktop
(kali@kali)-[~/Desktop]
$ nano BOF.c
(kali@kali)-[~/Desktop]
$ nano BOF.c
(kali@kali)-[~/Desktop]
$ gcc -g BOF.c -o BOF
(kali@kali)-[~/Desktop]
$ ./BOF
Si prega di inserire il nome utente:Miriana
Nome utente inserito: Miriana
(kali@kali)-[~/Desktop]
$ ./BOF
Si prega di inserire il nome utente:cybersecurityValerioGroup
Nome utente inserito: cybersecurityValerioGroup
zsh: segmentation fault ./BOF
```

6. Modifica del buffer per testare input più lungo

6.1 Modifica del codice

Modifica nel codice tramite i passaggi iniziali: `char buffer[30];`

Aumentiamo la dimensione del buffer per accettare input più lunghi.

```
kali@kali: ~/Desktop
File Azioni Modifica Visualizza Aiuto
GNU nano 8.4 BOF.c
#include <stdio.h>

int main () {
    char buffer[30];
    printf("Si prega di inserire il nome utente:");
    scanf("%s", buffer);
    printf("Nome utente inserito: %s\n", buffer);
    return 0;
}
```

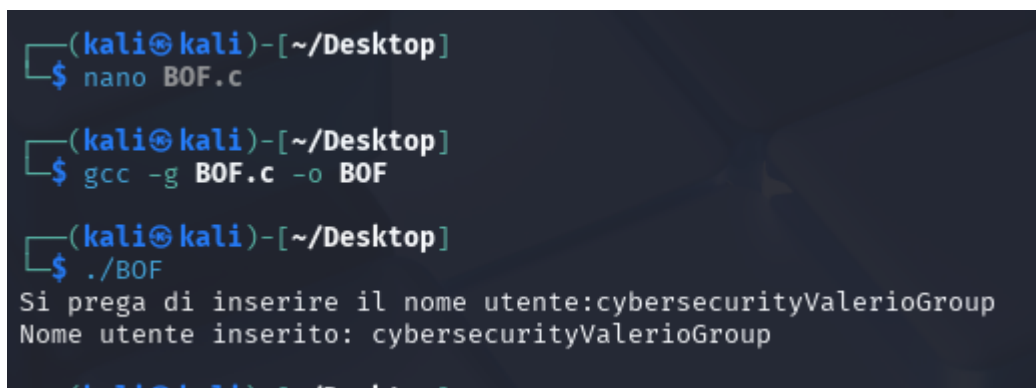
6.2 Ricompilazione

Comando: `gcc -g BOF.c -o BOF`

Applichiamo le modifiche e generare nuovo eseguibile.

6.3 Esecuzione con input entro 30 caratteri

Comando: `./BOF`



```
(kali㉿kali)-[~/Desktop]
$ nano BOF.c

(kali㉿kali)-[~/Desktop]
$ gcc -g BOF.c -o BOF

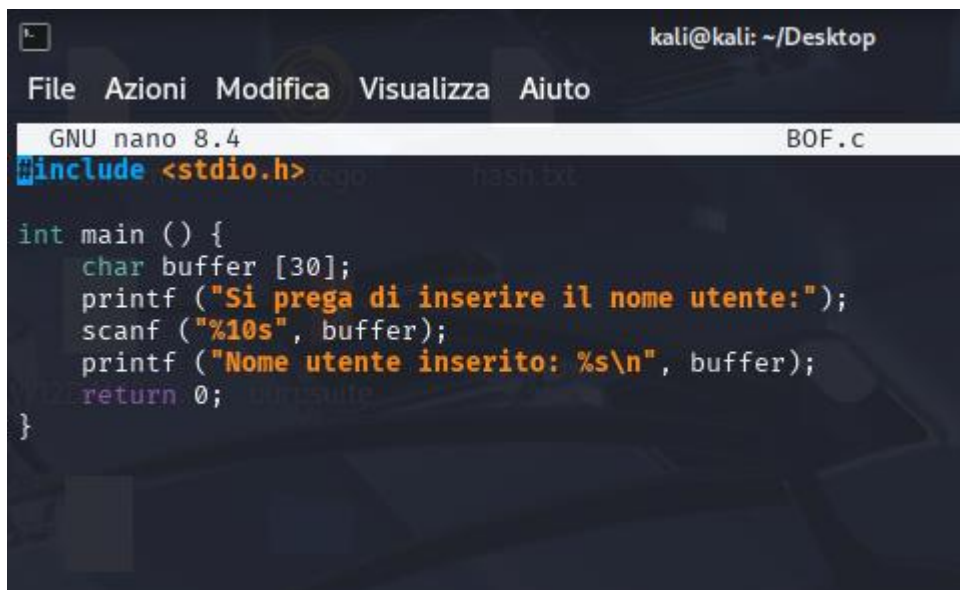
(kali㉿kali)-[~/Desktop]
$ ./BOF
Si prega di inserire il nome utente:cybersecurityValerioGroup
Nome utente inserito: cybersecurityValerioGroup
```

7. Prevenzione del Buffer Overflow (Parte Facoltativa)

7.1 Modifica del codice con controllo input

Modifica nel codice: `scanf("%10s", buffer);`

%10s limita l'input a 10 caratteri, evitando BOF anche se l'utente inserisce di più.
Finalità: Sanitizzazione dell'input direttamente nel formato scanf.



```
kali@kali: ~/Desktop
File Azioni Modifica Visualizza Aiuto
GNU nano 8.4 BOF.c
#include <stdio.h>

int main () {
    char buffer [30];
    printf ("Si prega di inserire il nome utente:");
    scanf ("%10s", buffer);
    printf ("Nome utente inserito: %s\n", buffer);
    return 0;
}
```

7.2 Test del programma modificato

Comando: ./BOF

Per completare la parte facoltativa dell'esercitazione, si interviene direttamente sul codice sorgente per rendere il programma più sicuro. La modifica consiste nel limitare la quantità di caratteri che l'utente può inserire, agendo sul formato della funzione scanf. In particolare, si sostituisce "%s" con "%10s", indicando al programma di accettare al massimo dieci caratteri, anche se l'utente ne digita di più. Questa semplice modifica rappresenta una forma di sanitizzazione dell'input, che protegge il buffer da scritture eccessive e impedisce il verificarsi di un buffer overflow.

Dopo aver salvato il file e ricompilato il programma, si esegue un nuovo test inserendo una stringa molto lunga, ben oltre i dieci caratteri consentiti. Il programma, però, si comporta correttamente: legge solo i primi dieci caratteri e ignora il resto, evitando qualsiasi errore di segmentazione. Il risultato conferma che il controllo sull'input è efficace e che il programma, pur ricevendo un input potenzialmente pericoloso, riesce a gestirlo in modo sicuro. Questa dimostrazione evidenzia quanto sia importante validare l'input utente per proteggere la memoria e garantire la stabilità del software.

```
(kali㉿kali)-[~/Desktop]
$ gcc -g BOF.c -o BOF

(kali㉿kali)-[~/Desktop]
$ ./BOF
Si prega di inserire il nome utente:CiaoValerioeccolapartefacoltativa
Nome utente inserito: CiaoValeri
```

8. Conclusione

Questa esercitazione ha permesso di esplorare in modo pratico una delle vulnerabilità più insidiose nei programmi scritti in C: il buffer overflow. Attraverso la scrittura, compilazione ed esecuzione di un codice volutamente vulnerabile, è stato possibile osservare come un input non controllato possa compromettere la stabilità del programma, generando un errore di segmentazione e potenzialmente aprendo la strada a exploit più gravi.

La parte facoltativa ha dimostrato quanto sia efficace anche una semplice modifica nella gestione dell'input per prevenire il problema. Limitando la lunghezza della stringa accettata da , il programma diventa immediatamente più sicuro, evitando scritture in memoria non autorizzata. Questo intervento, pur essendo minimale, rappresenta una buona pratica di programmazione difensiva e sottolinea l'importanza della validazione dell'input utente.

In sintesi, l'esercitazione non solo ha mostrato il funzionamento tecnico del buffer overflow, ma ha anche evidenziato il valore della consapevolezza e della prevenzione nella scrittura di codice sicuro. Comprendere questi meccanismi è fondamentale per chiunque voglia operare nel campo della sicurezza informatica o dello sviluppo software responsabile.